

RESOLVEHUB

Issue Tracking & Support Management System

Technical Project Report & Portfolio Documentation

Prepared as a concise record of the design, implementation, testing, security controls and deployment of ResolveHub.

Prepared by Roshitha [Gunasekaran]

Github: [roshitha05](#)

Table of Contents

1. Executive Summary	3
1.1 Project Links	3
2. Project Objectives	3
3. Scope	3
4. Technology Stack.....	3
5. System Design	4
5.1 Application Structure	4
5.2 Data Model.....	4
5.3 Authentication and Authorization.....	4
6. Implemented Features.....	4
6.1 Registration and Login.....	4
6.2 Dashboard	5
6.3 Issue Creation.....	7
6.4 Issue List, Search and Filtering	8
6.5 Issue Details and Status Update.....	11
7. Security Implementation.....	13
8. Testing and Quality Assurance	13
9. CI/CD and Deployment.....	15
10. Deployment Configuration	16
11. Challenges and Resolutions.....	16
12. Limitations and Constraints.....	16
13. Final System Verification	17
14. Future Improvements.....	17
15. Summary	17
Appendix A — Screenshots	18
Appendix B — Repository Evidence	19

1. Executive Summary

ResolveHub is a web-based issue tracking and support management system built as a practical full-stack portfolio project. The application focuses on a straightforward support workflow: a user creates an account, signs in, submits an issue, reviews their own requests, filters or searches them, opens an issue for more detail, and updates its status as work progresses.

The project was deliberately kept focused rather than overloaded with decorative features. The goal was to demonstrate a complete software delivery cycle: frontend development with React and TypeScript, persistent data and authentication with Supabase, database-level access control through Row Level Security (RLS), automated testing, production builds, source control, and automated deployment through GitHub Actions.

This report documents the implemented system and leaves clearly marked spaces for screenshots from the final deployed application. It is intended to complement the repository README rather than act as a substitute for the source code.

1.1 Project Links

Live Application:

<https://roshitha05.github.io/resolvehub/>

GitHub Repository:

<https://github.com/roshitha05/resolvehub>

2. Project Objectives

- Build a clean and responsive support issue interface using React and TypeScript.
- Store and retrieve real issue data using a PostgreSQL-backed Supabase project.
- Implement user registration, email verification, login, logout and persistent sessions.
- Protect application routes and enforce issue ownership at the database layer.
- Provide practical issue management features such as search, filtering, priority and status updates.
- Add automated tests and static code quality checks.
- Deploy the application through a repeatable CI/CD workflow rather than a manual upload.

3. Scope

ResolveHub is an MVP-oriented support management system. Its implemented scope includes authentication, a dashboard, issue creation, issue listing, search and filtering, issue details, priority labels, status updates, ownership-based access control, automated tests and deployment.

Features such as comments, attachments, notifications, charts and complex team assignment were intentionally excluded from the first version. This kept the project small enough to finish and test properly while still demonstrating the core engineering concerns expected in a real application.

4. Technology Stack

Area	Technology	Purpose
Frontend	React + TypeScript	Component-based UI and typed application logic
Build Tool	Vite	Local development and optimized production builds
Routing	React Router (HashRouter)	Client-side navigation compatible with

		GitHub Pages
Backend	Supabase	PostgreSQL database, authentication and API access
Security	Supabase RLS	Database-level ownership enforcement
Testing	Vitest + React Testing Library	Automated component and interaction testing
Code Quality	ESLint	Static analysis and consistency checks
Version Control	Git + GitHub	Source history and remote repository
CI/CD	GitHub Actions	Automated test, lint, build and deployment pipeline
Hosting	GitHub Pages	Public frontend deployment

5. System Design

5.1 Application Structure

The frontend is divided into reusable layout components and page-level components. The sidebar and header form the authenticated application shell, while pages handle dashboard data, issue creation, issue lists, issue details, login and registration. Supabase is initialized through a dedicated client module.

Hash-based routing was selected so that direct navigation remains compatible with GitHub Pages without requiring server-side route rewriting.

5.2 Data Model

The central application table is `public.issues`. Each record contains an identifier, title, description, priority, status, creation timestamp and `user_id`. The `user_id` references Supabase Auth users and connects every new issue to the account that created it.

Field	Type / Values	Role
<code>id</code>	<code>bigint</code>	Primary identifier
<code>title</code>	<code>text</code>	Short issue summary
<code>description</code>	<code>text</code>	Detailed issue information
<code>priority</code>	<code>low / medium / high</code>	Urgency classification
<code>status</code>	<code>open / in_progress / resolved</code>	Workflow state
<code>created_at</code>	<code>timestampz</code>	Creation timestamp
<code>user_id</code>	<code>uuid</code>	Owner linked to <code>auth.users</code>

5.3 Authentication and Authorization

Authentication is handled through Supabase Auth. Users can register with email and password, verify their email address, sign in, retain a session across refreshes, and sign out. Application routes are protected in React so unauthenticated users are redirected to the login page.

Frontend route protection is not treated as the security boundary. RLS policies on the `issues` table restrict `SELECT`, `INSERT` and `UPDATE` operations to authenticated users whose `auth.uid()` matches the row's `user_id`. This means a user cannot access another user's issues simply by changing frontend requests.

The `user_id` foreign key uses `ON DELETE CASCADE` so that deleting an account also removes its owned issues, preventing orphaned records.

6. Implemented Features

6.1 Registration and Login

The authentication screens use the same ResolveHub visual identity as the main application while remaining separate from the authenticated sidebar layout. Registration uses Supabase sign-up with an email verification

redirect, while login uses password authentication. Successful authentication updates the application session and unlocks protected routes.

Registration and Login

 ResolveHub

Welcome back

Sign in to manage your support requests.

Email

Password

Sign In

Don't have an account? [Create account](#)

Registration

 ResolveHub

Create account

Register to start managing support requests.

Email

Password

Create Account

Already have an account? [Sign in](#)

Screenshot of the deployed registration and login page.

6.2 Dashboard

The dashboard provides a compact overview of the user's issues. Counts are calculated from the complete issue dataset for Total, Open, In Progress and Resolved, while only the three most recent records are shown in the Recent Issues table. This avoids the earlier logic problem where limiting the query to three rows also produced incorrect totals.

Dashboard

 ResolveHub

Dashboard

Issues

New Issue

Dashboard

Overview of your support requests

Support User

Team Member

Logout

Welcome back

Here's an overview of your support issues.

Total Issues

0

Open

0

In Progress

0

Resolved

0

Recent Issues

Latest support requests submitted to ResolveHub.

+ Create Issue

ID	Issue	Priority	Status
No recent issues.			

Dashboard showing the four status cards and Recent Issues table.

6.3 Issue Creation

Authenticated users can create a support request by entering a title, description and priority. Before insertion, the application retrieves the authenticated user and stores the user's ID with the issue. The submit button includes a creating state and the form displays an error message if the operation fails.

Create New Issue

The screenshot shows the 'New Issue' page in the ResolveHub application. On the left is a dark sidebar with the ResolveHub logo and navigation links for Dashboard, Issues, and a highlighted 'New Issue' button. The main content area has a header with the title 'New Issue' and a subtitle 'Create a new support request'. In the top right corner, there is a user profile for 'Support User' (Team Member) and a 'Logout' button. Below the header, the section is titled 'Create New Issue' with the instruction 'Submit a support request for the team to review.' The form itself contains three input fields: 'Issue Title' with the text 'Unable to access account settings', 'Description' with the text 'The account settings page does not load after signing in. The page remains blank after multiple refresh attempts.', and 'Priority' set to 'Medium'. At the bottom right of the form are two buttons: 'Cancel' and '+ Create Issue'.

Issue creation form with sample, non-sensitive data.

6.4 Issue List, Search and Filtering

The Issues page retrieves the user's permitted records through Supabase and displays them in a table. Search checks both title and description text. Users can also filter by status and priority. The page includes loading, empty and error states so the interface does not silently fail when data is unavailable.

Issues: Search and Filters

ResolveHub

Dashboard

Issues

New Issue

Issues

View and manage your support issues

Support User

Team Member

Logout

All Issues

View, search and manage your support requests.

+ Create Issue

Search issues...

All Statuses

All Priorities

ID	Issue	Priority	Status	Created
#7	Improve mobile navigation	High	Open	16/09/2026
#6	Issue status not updating	High	Resolved	16/09/2026
#5	Update profile information	Low	Open	16/09/2026
#4	Dashboard loading slowly	High	In Progress	16/09/2026
#3	Unable to access account settings	Medium	Open	16/09/2026

ResolveHub

Dashboard

Issues

New Issue

Issues

View and manage your support issues

Support User

Team Member

Logout

All Issues

View, search and manage your support requests.

+ Create Issue

Search issues...

All Statuses

All Priorities

ID	Issue	Priority	Status	Created
#7	Improve mobile navigation	High	Open	16/09/2026

Relevant Word Search in Search Bar

ResolveHub

Dashboard

Issues

New Issue

Issues

View and manage your support issues

Support User

Team Member

Logout

All Issues

View, search and manage your support requests.

+ Create Issue

Search issues...

All Statuses

All Priorities

ID	Issue	Priority	Status	Created
#7	Improve mobile navigation	High	Open	16/09/2026

8

Irrelevant Word Search in Search Bar

 ResolveHub

Dashboard

Issues

New Issue

Issues

View and manage your support issues

Support User

Team Member

Logout

All Issues

View, search and manage your support requests.

Create Issue

Search issues...

All Statuses

All Priorities

ID	Issue	Priority	Status	Created
No issues found.				

Filtered: Status (Open)

 ResolveHub

Dashboard

Issues

New Issue

Issues

View and manage your support issues

Support User

Team Member

Logout

All Issues

View, search and manage your support requests.

Create Issue

Search issues...

Open

All Priorities

ID	Issue	Priority	Status	Created
#7	Improve mobile navigation	High	Open	16/09/2026
#5	Update profile information	Low	Open	16/09/2026
#3	Unable to access account settings	Medium	Open	16/09/2026

Filtered: Priority (High)

 ResolveHub

Dashboard

Issues

New Issue

Issues

View and manage your support issues

Support User
Team Member

Logout

All Issues

View, search and manage your support requests.

+ Create Issue

Search issues...

All Statuses

High

ID	Issue	Priority	Status	Created
#7	Improve mobile navigation	High	Open	16/09/2026
#6	Issue status not updating	High	Resolved	16/09/2026
#4	Dashboard loading slowly	High	In Progress	16/09/2026

Filtered: Status (Resolved) & Priority (High)

 ResolveHub

Dashboard

Issues

New Issue

Issues

View and manage your support issues

Support User
Team Member

Logout

All Issues

View, search and manage your support requests.

+ Create Issue

Search issues...

Resolved

High

ID	Issue	Priority	Status	Created
#6	Issue status not updating	High	Resolved	16/09/2026

Shows several issues and, with search and filter.

6.5 Issue Details and Status Update

Selecting an issue opens a dedicated detail route using the issue ID. The page displays the description, priority, current status and creation date. Status can be changed between Open, In Progress and Resolved. Updates are persisted through Supabase and remain after a page refresh.

Issue Details

ResolveHub

Dashboard

Issues

New Issue

Issue Details

View and manage this support request

Support User

Team Member

Logout

Issue #7

← Back to Issues

Improve mobile navigation

View and manage this support request.

Description

The navigation menu is difficult to use on smaller mobile screens and could be made more responsive.

Priority

High

Status

Open

Open

Created

16/09/2026

ResolveHub

Dashboard

Issues

New Issue

Issue Details

View and manage this support request

Support User

Team Member

Logout

Issue #7

← Back to Issues

Improve mobile navigation

View and manage this support request.

Description

The navigation menu is difficult to use on smaller mobile screens and could be made more responsive.

Priority

High

Status

Resolved

Resolved

Created

16/09/2026

Updated Issues Page

 ResolveHub

Dashboard

Issues

New Issue

Issues

View and manage your support issues

Support User
Team Member

Logout

All Issues

View, search and manage your support requests.

Search issues...

All Statuses

All Priorities

ID	Issue	Priority	Status	Created
#7	Improve mobile navigation	High	Resolved	16/09/2026
#6	Issue status not updating	High	Resolved	16/09/2026
#5	Update profile information	Low	Open	16/09/2026
#4	Dashboard loading slowly	High	In Progress	16/09/2026
#3	Unable to access account settings	Medium	Open	16/09/2026

Dashboard Updated

Dashboard

Overview of your support requests

Support User
Team Member

Logout

Welcome back

Here's an overview of your support issues.

Total Issues
5

Open
2

In Progress
1

Resolved
2

Recent Issues

Latest support requests submitted to ResolveHub.

Create Issue

ID	Issue	Priority	Status
#7	Improve mobile navigation	High	Resolved
#6	Issue status not updating	High	Resolved
#5	Update profile information	Low	Open

Insert a detail-page screenshot with the status control visible.

7. Security Implementation

The initial development version used temporary permissive policies to prove database connectivity. These were removed before deployment and replaced with authenticated ownership policies. This distinction is important: the final application does not rely on hidden buttons or frontend checks to prevent cross-user access.

Control	Implementation
Authentication	Supabase email/password authentication with verification
Route protection	Unauthenticated users are redirected to /login
Read access	RLS SELECT policy requires <code>auth.uid() = user_id</code>
Create access	RLS INSERT policy requires the new row to belong to <code>auth.uid()</code>
Update access	RLS UPDATE policy restricts changes to owned rows
User deletion	Foreign key uses ON DELETE CASCADE
Secrets	.env is excluded from Git; CI receives client configuration through GitHub repository secrets
Privileged keys	No service-role key is exposed in browser code

ResolveHub uses PostgreSQL Row Level Security (RLS) through Supabase to ensure that authenticated users can only access issues associated with their own account. For example, the following SELECT policy compares the authenticated user's ID with the `user_id` stored on each issue:

RLS Policy Screenshot

```
1 create policy "Users can view own issues"
2 on public.issues
3 for select
4 to authenticated
5 using (auth.uid() = user_id);
```

Code Example 1. RLS policy restricting issue visibility to the authenticated owner.

In this policy, `auth.uid()` returns the ID of the currently authenticated user. A row is returned only when this value matches the issue's `user_id`. Similar ownership checks are applied to INSERT and UPDATE operations, preventing users from reading or modifying issues that belong to another account.

8. Testing and Quality Assurance

The project uses Vitest with React Testing Library and user-event. The test suite mocks Supabase so tests are deterministic and do not modify the live database. The current suite focuses on meaningful user-facing behaviour rather than chasing a large test count.

Automated Test	Expected Behaviour
Render issues from Supabase response	Returned issue records appear in the table
Search filtering	Matching issue remains while non-matching issue is removed
Status filtering	Selecting Resolved shows resolved records and hides unrelated records

The completed test run reported 1 test file passed and 3 tests passed. A production build was also completed successfully, followed by ESLint. These checks are repeated automatically in the deployment workflow.

Automated Test Result

test-and-build summary

Vitest Test Report

Summary

- Test Files:  1 pass · 1 total
- Test Results:  3 passes · 3 total

Job summary generated at run-time

Successful Vitest Automated Test Results for ResolveHub .

9. CI/CD and Deployment

ResolveHub is deployed with GitHub Actions and GitHub Pages. A push to the main branch triggers a workflow that checks out the repository, configures Node.js, installs dependencies, runs automated tests, runs ESLint, creates a production build, uploads the dist artifact and deploys it to GitHub Pages.

The first CI run exposed a Node runtime compatibility issue in the test environment. The workflow was corrected to use Node.js 22, after which the pipeline completed successfully. Keeping this fix in the project history provides a realistic example of diagnosing a difference between local development and CI.

Successful Deployment

Deploy ResolveHub

deploy.yml

Filter workflow runs

3 workflow runs

Event Status Branch Actor

This workflow has a workflow_dispatch event trigger.

Run workflow

✓ fix: correct authentication redirect URL

Deploy ResolveHub #3: Commit c5c1060 pushed by roshitha05

main

36 minutes ago

43s

✓ fix: update CI Node version

Deploy ResolveHub #2: Commit 898b80e pushed by roshitha05

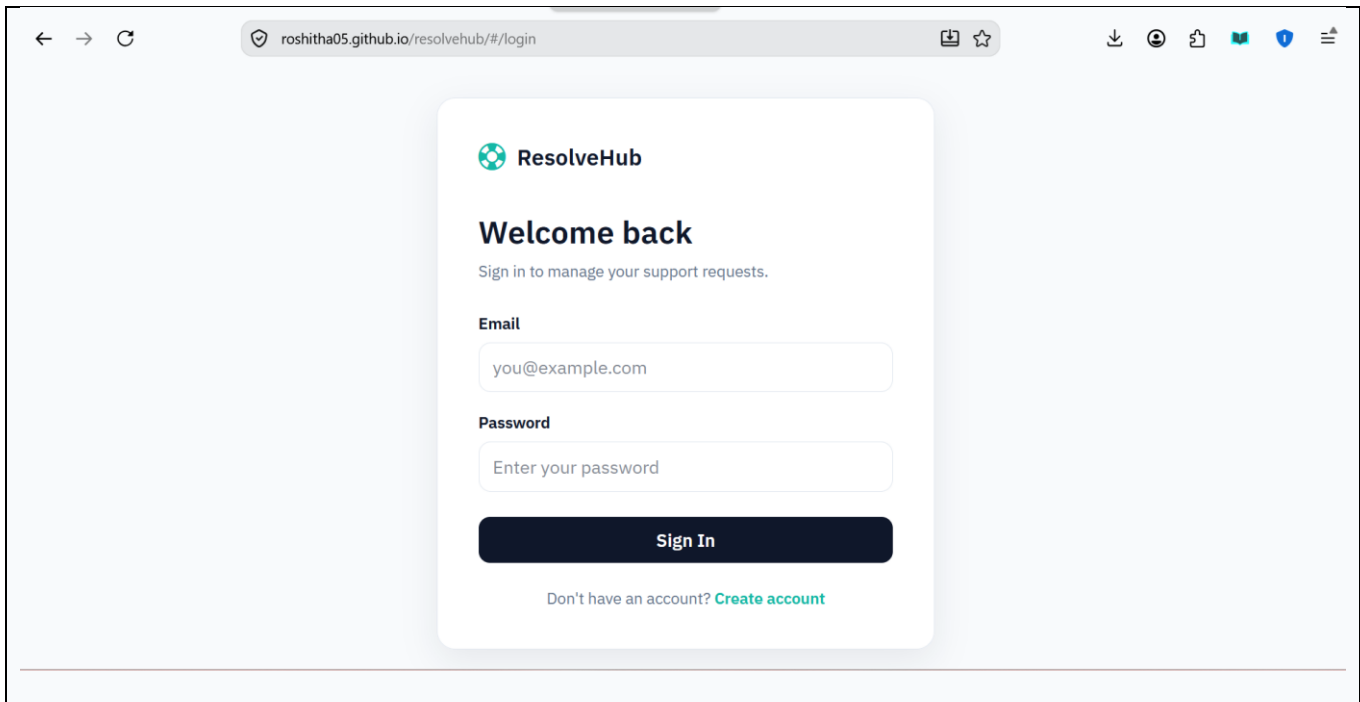
main

Today at 9:15 PM

50s

Successful ResolveHub Deployment Through GitHub Actions.

Live GitHub Pages Application



ResolveHub Application Successfully Deployed on GitHub Pages.

10. Deployment Configuration

Vite is configured with the /resolvehub/ base path so generated assets resolve correctly when the application is served from the GitHub Pages repository subdirectory. Supabase authentication URL configuration should include the permanent GitHub Pages address so email verification returns users to the deployed application.

The Supabase URL and publishable client key are supplied to GitHub Actions as repository secrets. These values are used during the frontend build. A Supabase service-role key is not used in the browser application.

11. Challenges and Resolutions

Challenge	Resolution
Status appeared to update but reverted after refresh	Identified missing UPDATE RLS permission; added an appropriate development policy, then replaced it with ownership-based authenticated RLS.
Issue details showed the Dashboard heading	Updated the header logic to detect /issues/:id routes and display an Issue Details heading.
Dashboard totals could become inaccurate	Removed the three-row query limit for statistics and sliced the already-fetched dataset only for the Recent Issues table.
In Progress badge styling did not match	Mapped the database value in_progress to the existing status-progress CSS class.
Email verification reached an error page	Configured an explicit email redirect and prepared the permanent deployed URL for Supabase Auth URL configuration.
GitHub Actions tests failed although local tests passed	Traced the CI error to runtime compatibility and updated the workflow from Node.js 20 to Node.js 22.

12. Limitations and Constraints

ResolveHub was developed as a focused portfolio project using free-tier services. The system prioritises the core issue-management workflow rather than attempting to replicate the full functionality of an enterprise helpdesk platform.

The current version supports individual issue ownership and does not include support-agent assignment, comments, file attachments, notifications or advanced role-based permissions. Authentication and database services are provided through Supabase, meaning the application is subject to the usage and email-delivery limits of the selected service tier. The frontend is hosted through GitHub Pages and uses client-side hash routing to support navigation without a dedicated application server.

These constraints were considered appropriate for the project scope, allowing the implementation to focus on full-stack development, secure database access, automated testing and CI/CD deployment.

13. Final System Verification

Test	Result
User registration and email verification	Pass
User login and protected route access	Pass
Issue creation with different priorities	Pass
Dashboard statistics	Pass
Issue search by title/description	Pass
Status and priority filtering	Pass
Issue status update	Pass
Status persistence after refresh	Pass
Logout and protected-route redirection	Pass
Vitest automated tests	Pass
Production build and ESLint	Pass
GitHub Actions CI/CD deployment	Pass

14. Future Improvements

The current version is intentionally an MVP. Reasonable future extensions would include role-based support staff access, issue assignment, comments, attachments, audit history, pagination, better accessibility testing, broader automated test coverage and server-side functions for privileged operations. These were left outside the current scope so the deployed version remained complete and maintainable.

15. Summary

ResolveHub demonstrates more than a static React interface. It combines typed frontend development, client-side routing, authentication, relational data, database-level authorization, automated testing, source control and CI/CD in one small application. The project also records several practical debugging decisions, including RLS persistence, route-aware headers, accurate dashboard aggregation and CI runtime compatibility.

For a portfolio reviewer, the strongest evidence is the combination of the live application, source repository, automated tests, green deployment workflow and ownership-based RLS configuration. Together, these show that the project was taken through implementation, verification and deployment rather than stopping at a local prototype.

Appendix A — Screenshots

Figure	Recommended Screenshot	What It Demonstrates
1	Login or registration	Supabase authentication UI
2	Dashboard	Issue totals and recent activity
3	Create Issue	Form workflow and priority selection
4	Issues with search/filter active	Client-side interaction
5	Issue Details	Read/update workflow
6	3/3 Vitest result	Automated testing evidence
7	Green GitHub Actions run	CI/CD evidence
8	Live GitHub Pages app	Production deployment

Appendix B — Repository Evidence

- Live GitHub Pages deployment.
- Source repository and main application structure.
- `src/pages/issues.test.tsx` for the automated tests.
- `.github/workflows/deploy.yml` for the CI/CD workflow.
- Supabase RLS policy summary (document the policy logic; do not expose privileged credentials).
- Commands: `npm run test:run`, `npm run lint` and `npm run build`.